

Job Sequencing Problem

Last Updated : 23 Jul, 2025



Given two arrays: **deadline[]** and **profit[]**, where the **index of deadline[]** represents a job ID, and **deadline[i]** denotes the deadline for that job and **profit[i]** represents profit of doing **ith** job. **Each job takes exactly one unit of time to complete**, and only one job can be scheduled at a time. A job earns its corresponding profit **only if it is completed within its deadline**.

The objective is to determine:

1. The **maximum profit** that can be obtained by scheduling the jobs optimally.
2. The **total number of jobs completed** to achieve this maximum profit.

Examples:

Input: `deadline[] = [4, 1, 1, 1], profit[] = [20, 10, 40, 30]`

Output: 2 60

Explanation: We select 1st and 3rd jobs. All jobs except first job have a deadline of 1, thus only one of these can be selected along with the first job with the total profit gain of $20 + 40 = 60$.

Input: `deadline[] = [2, 1, 2, 1, 1], profit[] = [100, 19, 27, 25, 15]`

Output: 2 127

Explanation: The first and third job have a deadline of 2, thus both of them can be completed and other jobs have a deadline of 1, thus any one of them can be completed. Both the jobs with a deadline of 2 is having the maximum associated profit, so these two will be completed, with the total profit gain of $100 + 27 = 127$.

Table of Content

- [\[Naive Approach\] Using Greedy Approach and Sorting - \$O\(n^2\)\$ Time and \$O\(n\)\$ Space](#)
- [\[Expected Approach\] Using Greedy Approach, Sorting and Priority Queue - \$O\(n \cdot \log\(n\)\)\$ Time and \$O\(n\)\$ Space](#)
- [\[Alternate Approach\] Using Disjoint Set - \$O\(n \cdot \log\(d\)\)\$ Time and \$O\(d\)\$ Space](#)

[Naive Approach] Greedy Approach and Sorting - $O(n^2)$ Time and $O(n)$ Space

Step by Step implementation:

- **Store jobs as pairs of (Profit, Deadline):** Since we need to prioritize jobs with higher profits, we pair the profit and deadline together.
- **Sort Jobs Based on Profit:** We sort the jobs array in descending order of **profit** so that we prioritize scheduling the most profitable jobs first.
- **Create a Slot Array:** We create a `slot[]` array of size `n` (equal to the number of jobs) initialized with **zeroes**. This array will help track which time slots are occupied.
- **Iterate Over Each Job and Try to Schedule It:**
 - For each job, check if it can be placed in an available time slot.
 - The job should be **scheduled as late as possible but before its deadline.**
 - If an empty slot is found, schedule the job there, **increment the job count**, and **add its profit to the total.**
 - After processing all jobs, return the **number of jobs completed** and the **total profit earned.**

C++

Java

Python

C#

JavaScript

```
// C++ program to solve job sequencing
// problem with maximum profit
#include<bits/stdc++.h>
using namespace std;

vector<int> jobSequencing(vector<int> &deadline,
                        vector<int> &profit) {
    int n = deadline.size();

    // total job count which is done
    int cnt = 0;

    // total profit earned
    int totProfit = 0;

    // pair the profit and deadline of
```

```

// all the jos together
vector<pair<int, int>> jobs;
for (int i = 0; i < n; i++) {
    jobs.push_back({profit[i], deadline[i]});
}

// sort the jobs based on profit
// in decreasing order
sort(jobs.begin(), jobs.end(),
      greater<pair<int, int>>());

// array to check time slot for job
vector<int> slot(n, 0);
for (int i = 0; i < n; i++) {
    int start = min(n, jobs[i].second) - 1;
    for (int j = start; j >= 0; j--) {

        // if slot is empty
        if (slot[j] == 0) {
            slot[j] = 1;
            cnt++;
            totProfit+= jobs[i].first;
            break;
        }
    }
}

return {cnt, totProfit};
}

int main() {
    vector<int> deadline = {2, 1, 2, 1, 1};
    vector<int> profit = {100, 19, 27, 25, 15};
    vector<int> ans = jobSequencing(deadline, profit);
    cout<<ans[0]<<" "<<ans[1];
    return 0;
}

```

the order of insertion is swapped

Output

2 127

[Expected Approach] Greedy Approach, Sorting and Priority Queue - $O(n \cdot \log(n))$ Time and $O(n)$ Space

The main idea is to **sort** the jobs based on their deadlines in **ascending order**. This ensures that jobs with **earlier deadlines** are processed first, preventing situations where a job with a short deadline remains unscheduled because a job with a later deadline was chosen instead. We use a **min-heap** to keep track of the selected jobs, allowing us to efficiently replace lower-profit jobs when a more profitable job becomes available.

Step by Step implementation:

- Store jobs as pairs of (Deadline, Profit).
- **Sort Jobs Based on Deadline:** We sort the jobs array in ascending order of **deadline** so that we prioritize jobs with earlier deadlines are considered first.
- For each job (deadline, profit) in the sorted list:
 - If the job can be scheduled within its deadline (i.e., the number of jobs scheduled so far is less than the deadline), push its profit into the heap.
 - If the heap is full (equal to deadline), replace the existing lowest profit job with the current job if it has a higher profit.
 - This ensures that we always keep the most profitable jobs within the available slots.
- Traverse through the heap and store the total profit and the count of jobs.

C++ Java Python C# JavaScript

```
// C++ program to solve job sequencing
// problem with maximum profit

#include<bits/stdc++.h>
using namespace std;

vector<int> jobSequencing(vector<int> &deadline, vector<int>
&profit) {
    int n = deadline.size();
    vector<int> ans = {0, 0};

    // pair the profit and deadline of
```

```

// all the jos together
vector<pair<int, int>> jobs;
for (int i = 0; i < n; i++) {
    jobs.push_back({deadline[i], profit[i]});
}

// sort the jobs based on deadline
// in ascending order
sort(jobs.begin(), jobs.end());

// to maintain the scheduled jobs based on profit
priority_queue<int, vector<int>, greater<int>> pq;

for (const auto &job : jobs) {

    // if job can be scheduled within its deadline
    if (job.first > pq.size())
        pq.push(job.second);

    // Replace the job with the lowest profit
    else if (!pq.empty() && pq.top() < job.second) {
        pq.pop();
        pq.push(job.second);
    }
}

while (!pq.empty()) {
    ans[1] += pq.top();
    pq.pop();
    ans[0]++;
}

return ans;
}

int main() {
    vector<int> deadline = {2, 1, 2, 1, 1};
    vector<int> profit = {100, 19, 27, 25, 15};
    vector<int> ans = jobSequencing(deadline, profit);
    cout<<ans[0]<<" "<<ans[1];
    return 0;
}

```

Output

2 127

[Alternate Approach] Using Disjoint Set - $O(n * \log(d))$ Time and $O(d)$ Space

The job sequencing can also be done using [disjoint set](#) based on the maximum deadline of all the jobs. This approach is explained in article [Job Sequencing - Using Disjoint Set](#).

 Comment

More info 

Advertise with us

Similar Reads

Basics & Prerequisites



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information